

Sisteme de operare

Laborator 3

Semestrul I 2025-2026

Laborator 3

- file I/O (cont.)
- tipuri de fisiere
- permisiuni de acces la fisiere
- partajarea fisierelor
- lucrul cu directoare

Tipuri de fișiere

- fișiere obisnuite (*regular files*): contin date (text sau binare)
- directoare: contin numele altor fișiere si informatii despre ele
 - pot fi citite de catre procesele care au permisiunile potrivite
 - DOAR kernelul poate scrie in ele !
- fișiere speciale caracter: pt device-uri caracter (ex: tty, seriala)
- fișiere speciale block: discuri
- Obs: orice device din sistem e fie fisier block fie caracter
- FIFO: named pipe, mecanism IPC
- socket: abstractie pentru IPC peste retea
- link-uri simbolice: fisier care refera un alt fisier

ln -s <fisier-sursa> <link-simbolic>

ln -s /etc/profile ~/.system-wide-profile

Stat/fstat/lstat

- furnizeaza informatii despre un fisier

int stat(const char pathname, struct stat *buf);*

*int fstat(int fd, struct stat *buf);*

int lstat(const char pathname, struct stat *buf);*

- ex: *struct stat buf;*

lstat("/etc/profile", &buf);

printf("dimensiunea fisierului /etc/profile este %d\n", buf.st_size);

Obs: pt link-uri simbolice, dimensiunea fisierului este lungimea numelui fisierului referit de link

Set UID, set GID

- fiecare proces are asociat
 - UID, GID real: identitatea reala a utilizatorului provenita din */etc/passwd*
 - UID/GID efectiv
 - set-UID, set-GID salvate (copii ale UID/GID efectiv)
- in mod normal, UID/GID real = UID/GID efectiv
- cand se executa un program exista posibilitatea de a seta un flag in *st_mode* care spune ca:

“pe durata executiei acestui program UID/GID efectiv al procesului devine UID/GID-ul proprietarului fisierului executabil”

ex: comanda de schimbare a parolei utilizator

\$ passwd

/usr/bin/passwd este un program set-UID la root pt a avea drepturi de scriere in */etc/shadow*

Permisuni de acces la fisiere

- codificate in campul *st_mode* alt structurii *stat*
- grupate in trei categorii
 - permisiuni utilizator: S_IRUSR, S_IWUSR, S_IXUSR
 - permisiuni grup: S_IRGRP, S_IWGRP, S_IXGRP
 - permisiuni pt. alti utilizatori: S_IROTH S_IWOTH, S_IXOTH
- modificabile din shell cu ajutorul comenzii *chmod*

ex:

chmod u+rw <fisier>, chmod g-x <fisier>, chmod o-rwx <fisier>

sau

chmod 755 <fisier>, chmod 644 <fisier> , etc

- accesul la un fisier: conditionat de combinatia dintre valoarea UID-ului efectiv, respectiv GID-ului efectiv, al celui care executa comanda si bitii de permisiune

Acess/umask

- pt a testa accesibilitatea utilizatorului bazata pe UID/GID reale

*int access(const char *pathname, int mode)*

mode: “OR” logic intre valorile R_OK, W_OK, X_OK, F_OK

- Obs: *access* foloseste ID-urile reale !
- orice fisier nou creat are setata o masca implicita a permisiunilor, data de valoarea *umask* (comanda shell)

mode_t umask(mode_t cmask);

cmask e un OR intre bitii de permisiune

- bitii setati in *umask* sunt off in permisiunea noului fisier creat

Chmod/fchmod

- permit schimbarea permisiunilor de acces la fisiere

*int chmod(const char *pathname, mode_t mode)*

int fchmod(int fd, mode_t mode)

Stergerea fisierelor

- un fisier poate avea mai multe intrari de director care refera acelasi *i-node*
 - s.n. *linkuri hard*
 - create cu

*int link(const char *pathname, const char *newpath)*

- pt. a sterge o intrare de fisier dintr-un director se foloseste *unlink*

*int unlink(const char *pathname)*

- permisiuni necesare stergerii unui fisier dintr-un director
 - permisiune de scriere in director
 - permisiune de cautare in director (bitul *x* de executie setat)
 - Obs: un fisier se sterge efectiv de pe disc odata cu stergerea ultimului link !
- pt a sterge directoare se foloseste *rmdir*
 - Obs: un director nu se poate sterge decat daca e GOL !
- *remove* pt parametru fisier e *unlink*, iar pt director *rmdir*

*int remove(const char *pathname)*

Link-uri simbolice

- limitările linkurilor hard
 - link-ul si fisierul linkat trebuie sa se afle pe acelasi sistem de fisiere
 - doar *root-ul* poate crea linkuri hard catre directoare
- *linkurile simbolice* sunt fisiere care contin numele fisierului referit (un string)
 - Obs: fisierul referit poate sa nu existe ! (*broken link*)
- in general, comenzile shell dereferentiaza linkul simbolic
 - exceptii: *lstat*, *remove*, *rename*, *unlink*, *stat*
- creare link simbolic

*int symlink(const char *actualpath, const char *sympathy)*

- citire link simbolic

*int readlink(const char *pathname, char * buf, int bufsize)*

deschide link-ul si citeste numele fisierului referit de link (actiune combinata *open + read + close*)

Lucrul cu directoare

- create cu *mkdir*, sterse cu *rmdir*

*int mkdir(const char *pathname, mode_t mode)*

*int rmdir(const char *pathname)*

- Obs:
 - *mode* in apelul *mkdir* trebuie sa includa permisiunea de execute !
 - *rmdir* nu poate sterge decat un director GOL !
- citirea directoarelor: secventa *openddir* + *readdir* + *closedir*

*DIR *openddir(const char *pathname)*

*struct dirent *readdir(DIR *dp)*

*closedir(DIR *dp);*

Lucrul cu directoare (cont.)

- schimbarea directorului curent

*int chdir(const char *pathname)*

int fchdir(int fd)

- aflarea directorului de lucru current (current working directory)

*char *getcwd(char *buf, size_t size)*

Sisteme de operare

Laborator 3

File I/O (cont.)

1. Creati un nou subdirector *lab3/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab3/bin*.

2. Scrieti un program C **myfile.c** care emuleaza comanda *file*. Mai exact, programul primeste o serie de fisiere ca argumente in linie de comanda si afiseaza ce tip de fisiere sunt. De exemplu puteti folosi urmatoarea linie de comanda:

```
$ myfile /home myfile.c /dev/tty1 /dev/sda1 my_symbolic_link myfifo
```

Indicatie: folositi apelul sistem *lstat* si comenzile *ln* (pentru a crea cu flag-ul *-s* un link simbolic in directorul curent, de pilda) si *mknod/mkfifo* (pentru a crea in directorul curent, de pilda, un fisier special de tip FIFO).

3. Scrieti un program C **temp.c** care simuleaza felul in care programele complexe gestioneaza fisierele temporare. In acest sens, mai intai creati un fisier *tempfile* cu urmatoarea comanda:

```
$ echo "this is a temporary file" > tempfile
$ cat tempfile
$ ls -l tempfile
```

Programul deschide fisierul *tempfile* folosind apelul sistem *open* apoi sterge fisierul *tempfile* folosind apelul sistem *unlink* si afiseaza pe ecran un mesaj din care sa reiasa ca fisierul *tempfile* a fost sters. Apoi simuleaza executarea unui task de 15 secunde apeland functia de biblioteca *sleep* (*man 3 sleep*). Dupa ce programul iese din *sleep*, citeste continutul fisierului *tempfile*, il afiseaza pe ecran si termina executia.

Pentru a intelege mai bine ce se intampla, rulati programul in background (folosind "&"). De exemplu, daca programul se numeste **temp.c** si va aflati in subdirectorul *src*:

```
$ gcc -o ../bin/temp temp.c
$ temp &
```

Dupa ce programul a afisat pe ecran notificarea stergerii fisierului *tempfile*, rulati comanda

```
$ ls -l tempfile
```

pentru a va convinge ca programul a fost intr-adevar sters. Asteptati 15 secunde. Ce se intampla? Cum va explicati efectul executiei programului daca reflectati la suita de operatii executate?

4. Scrieti un program C **myls.c** care afiseaza continutul unui director furnizat ca parametru al programului, simuland functionarea simpla, neparametrizata, a comenzii *ls*.

Indicatie: Folositi apelurile sistem *opendir/readdir/closedir*.

Folositi ulterior apelurile sistem *stat/fstat/lstat* pentru a implementa formatul lung al comenzii:

\$ myls -l <dirname/filename>

5. Scrieti un program C **mycd.c** care primeste un director ca parametru si schimba directorul curent la valoarea parametrului primit folosind *chdir*. Apoi, foloseste apelul sistem *getcwd* pentru a afla directorul curent si il tipareste pe ecran.

Testati programul utilizand urmatoarea secventa de comenzi:

```
$ pwd
$ mycd /etc
$ pwd
```

Ce observati? De ce e *cd* comanda interna a shell-ului?

6. Scrieti doua programe **ping.c** si **pong.c** care comunica printr-un fisier FIFO. Programele deschid fisierul FIFO atat pentru scriere cat si pentru citire. Programul **ping.c** scrie mesajul "*ping\n*" in fisier, doarme 5 secunde folosind apelul de biblioteca *sleep (man 3 sleep)*, si apoi citeste din FIFO raspunsul programului **pong.c** pe care il afiseaza pe ecran. Programul **pong.c** citeste un mesaj din FIFO pe care il afiseaza pe ecran iar apoi raspunde cu mesajul "*pong\n*".

Indicatie: Pentru a crea fisierul FIFO folositi comenzile *mknod/mkfifo*.